

Backend Basics

Git, GitHub, the command line, Node and Express

Prepared for Madoka Thomson

This is a self-study guide to go along with our lessons. Work through it at your own pace, and try the homework at the end before our next session. Everything here is meant to be copied, run, and experimented with, so don't worry about breaking anything.

Important Instruction: Before you begin with the code examples in Section 4, **you need to create a new file named `index.js` and copy the provided code into it.** Doing this will make the code much easier to understand, run, and experiment with!

1. Setting Up GitHub

GitHub is where your code lives once you want to save it, share it, or show it to someone else, like a future employer. Think of it as a backup and a portfolio at the same time.

Creating your account

1. Go to github.com and click Sign Up.
2. Choose a username you'd be comfortable putting on a resume. Keep it simple and professional.
3. Verify your email address when prompted.
4. The free plan is all you'll need for now.

What is a commit?

A commit is a saved snapshot of your project at a specific point in time. Every time you make meaningful progress, changing a file, adding a feature, fixing something, you take a snapshot. Over time, this builds a history you can look back through, compare, or undo if something breaks.

Git is the tool that manages these snapshots on your computer. GitHub is the website that stores a copy of them online.

Starting a new project

Inside any project folder, this command turns it into a Git project:

```
git init
```

You only run this once, right at the start of a project.

Saving changes with a commit

Whenever you've made changes you want to save, three commands do the job:

```
git add .  
git commit -m "describe what you changed here"  
git push
```

- `git add .` stages everything you've changed, telling Git "include all of this in the next snapshot."
- `git commit` actually takes the snapshot, with a short message describing what changed.
- `git push` sends that snapshot up to GitHub.

A good habit: Commit often, and write messages that would make sense to you a month from now, like "add greeting endpoint" rather than "update."

2. The Command Line

The command line is a text-based way of controlling your computer, typing instructions instead of clicking icons. It feels unfamiliar at first, but almost every developer tool, including Git, expects you to use it. Once it's comfortable, it's usually faster than clicking through folders.

Three commands to get started with:

- **mkdir** : Creates a new folder. For example, `mkdir my-project` creates a folder called my-project in your current location.
- **touch** : Creates a new, empty file. For example, `touch index.js` creates an empty file called index.js. On Windows, if touch isn't available, typing `nu1 > index.js` does the same thing.
- **ls** : Lists everything inside your current folder, so you can see what's there. On Windows, the equivalent command is `dir`.

Try this sequence yourself: make a folder, move into it, create a file, then list what's inside, just to get comfortable with the flow.

3. What Are Node and Express?

JavaScript was originally built to run only inside a browser, making buttons clickable and pages interactive. Node.js changed that. It's a runtime that lets JavaScript run outside the browser, so the same language can now power a server too.

Node by itself is quite low level. To handle different web addresses, read incoming data, and send back responses properly, you end up writing a lot of repetitive setup code yourself.

Express is a small framework built on top of Node that removes that repetition. It gives you simple, readable ways to say "when someone visits this address, run this function."

A simple way to think about it

If you're making a sandwich, you could grow the wheat, mill the flour, bake the bread, and make your own mayonnaise from scratch every single time. Or, you could buy a loaf of bread and a jar of mayonnaise, and just focus on assembling the sandwich.

Node gives you the raw ingredients and a kitchen. Express is the store-bought mayonnaise, a shortcut that gets you the same result with far less effort, so you can spend your time on the actual thing you're building instead of repetitive groundwork.

4. Code Examples

Example 1: Plain Node

This version uses only Node, no framework. Notice how much manual setup each route needs.

```
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.url === '/' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Welcome</h1>');
  } else if (req.url === '/about' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>About Us</h1>');
  } else if (req.url === '/api/user' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ name: 'Madoka', role: 'Analyst' }));
  } else {
    res.writeHead(404, { 'Content-Type': 'text/html' });
    res.end('<h1>Not Found</h1>');
  }
});

server.listen(3000, () => console.log('Server running on port 3000'));
```

Every route needs its own manual check, its own header, and JSON has to be converted by hand with `JSON.stringify`. Express exists to remove exactly this kind of repetition.

Example 2: Node with Express

Same idea, four routes this time, written with Express.

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('<h1>Welcome</h1>');
});

app.get('/about', (req, res) => {
  res.send('<h1>About Us</h1>');
});

app.get('/api/user', (req, res) => {
  res.json({ name: 'Madoka', role: 'Analyst' });
});

app.get('/api/orders/:id', (req, res) => {
  res.json({ orderId: req.params.id, status: 'pending' });
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

No manual if-chains, no manual headers, and JSON is handled automatically with `res.json()`. The last route reads a value directly from the web address itself, using `req.params.id`. This is called a dynamic route, since the response changes depending on what's typed after `/api/orders/`.

5. Installing Node, npm and Express

npm comes bundled with Node, so installing Node gives you both.

On Windows

1. Go to nodejs.org and download the LTS installer, then run it with the default options.
2. Open Command Prompt or PowerShell and confirm the install:

```
node -v
npm -v
```

3. Inside your project folder, set up the project and install Express:

```
npm init -y
npm install express
```

On Mac

1. Go to nodejs.org and download the LTS installer for macOS, then run it.
2. Alternatively, if you use Homebrew, run `brew install node` in Terminal.
3. Confirm the install:

```
node -v
npm -v
```

4. Inside your project folder, set up the project and install Express:

```
npm init -y
npm install express
```

Both commands should print a version number. If they do, you're ready to run the code examples above with `node index.js`.

6. Homework

Using the Express example as a reference, add the following routes to your own project. Try writing them yourself before looking back at the examples.

- **Greeting route:** `GET /api/greet/:name` should read the name from the address and return a JSON message greeting that person, for example `{ "message": "Hello, Madoka" }`.
- **About me route:** `GET /contact` should return a small piece of HTML with a few lines about yourself, your name, your background, and what you're currently learning.
- **Status route:** `GET /api/status` should return a JSON object describing the state of your imaginary service, for example `{ "status": "ok", "uptime": "2 hours" }`.

If you finish early

- **Skills route:** `GET /api/skills` should return a JSON list of a few skills you have, as an array of strings.
- **Search route:** `GET /api/search` should read a value from the address using a query string, for example `/api/search?q=java`, and return it back in the response. This introduces a new idea called a query parameter, read using `req.query.q` instead of `req.params`. Don't worry if this one takes some trial and error, we'll go over it properly next lesson.

Bring whatever you get working to our next session, even if it's not finished. We'll go through it together.